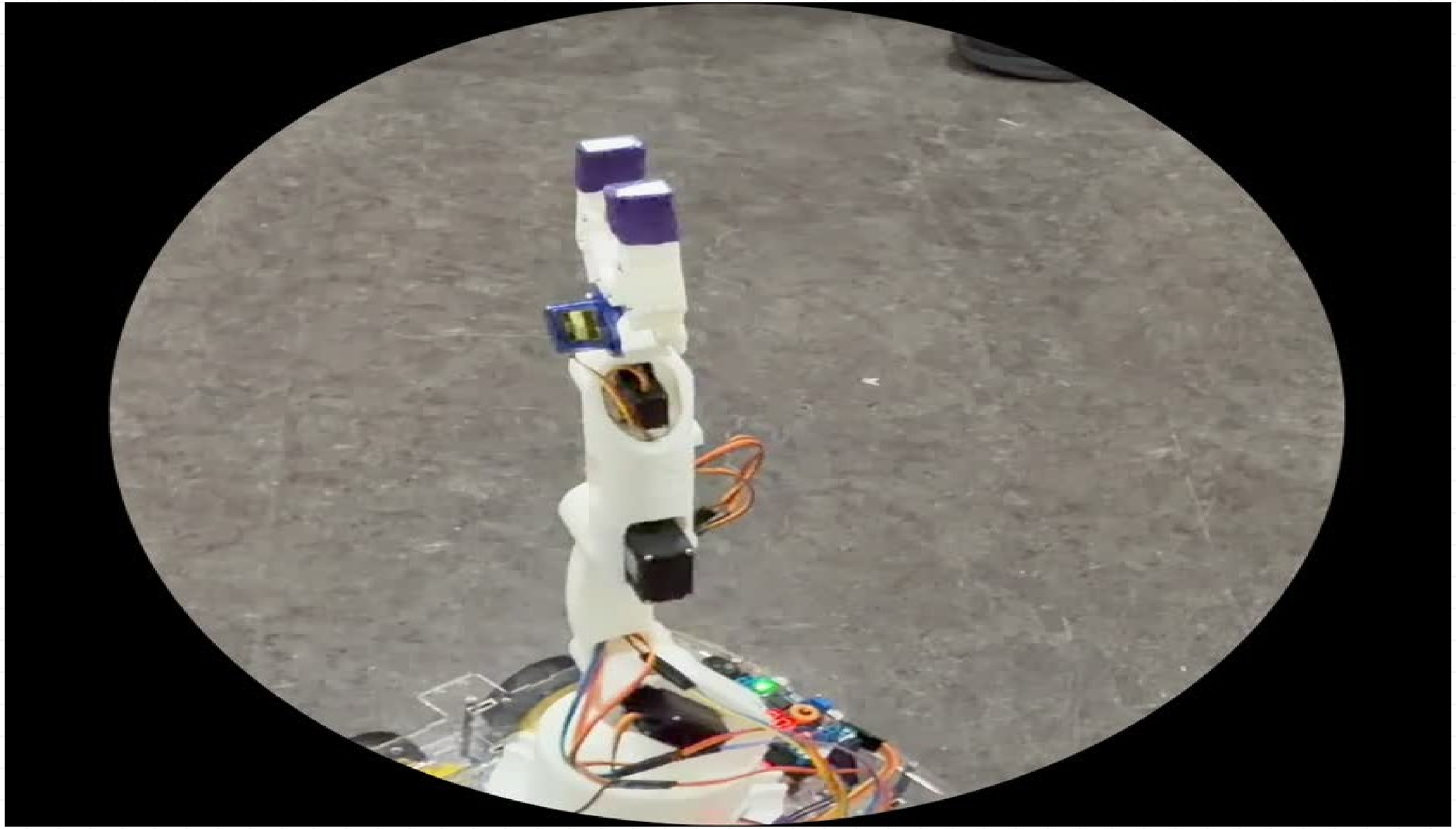
로봇 SW 개발자 3 기

임베디드 프로젝트

TEAM : 차팔이

정승호 전민준 이종현 채윤식 안재권



Contents

목차

01 프로젝트 소개

02 시스템 구성

03 시연 영상

04 핵심 기술

05 문제해결 및 발전

06 소감

01 프로젝트 소개

01 프로젝트 소개 _ 배경

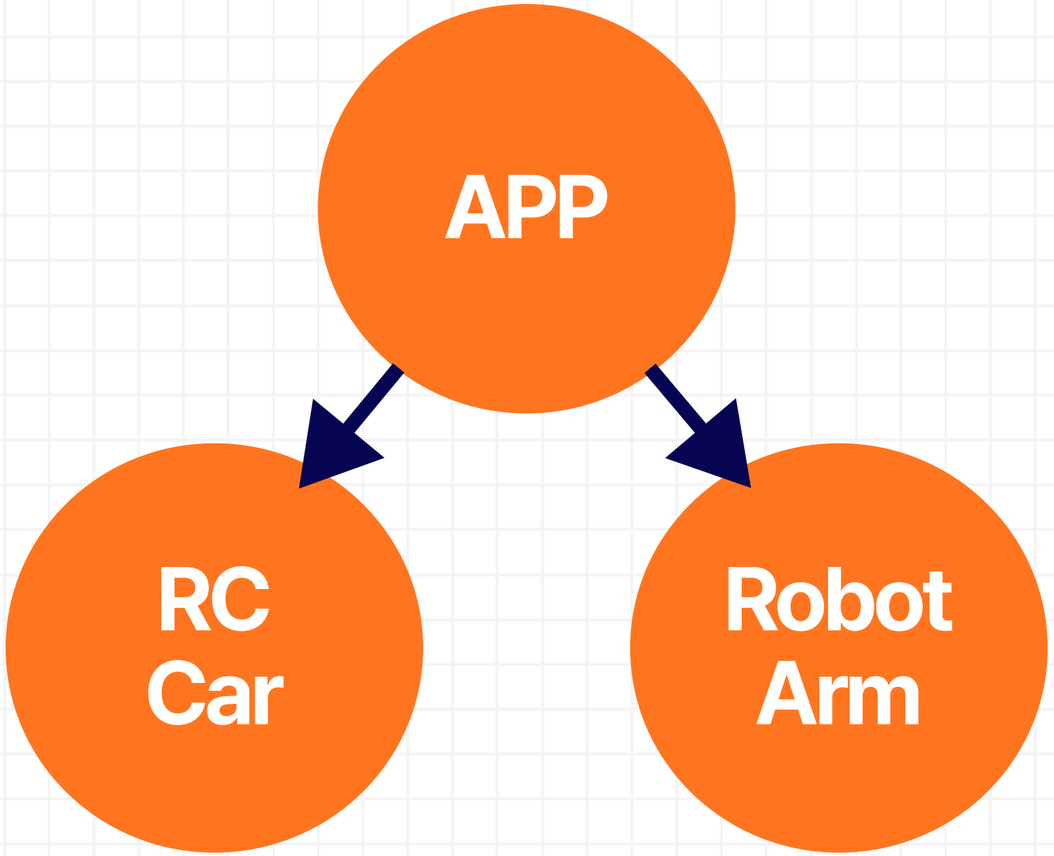
스마트 팩토리 / 물류 현장 등 산업 전반 통합 로봇 플랫폼 요구 증가
고정형 로봇팔의 작업 공간 한계 및 수송체와 로봇팔 간 통신 파편화
글로벌 MOMA 시장 연평균 21.4% 성장 전망 (2024 년 $5.29 억)

↓

고정형 로봇팔 + 이동성을 부여하고 통신 프로토콜 일원화
이동성 (Mobility) + 작업성 (Manipulation) 의 결합인
MOMA(Mobile Manipulator) 의 개발

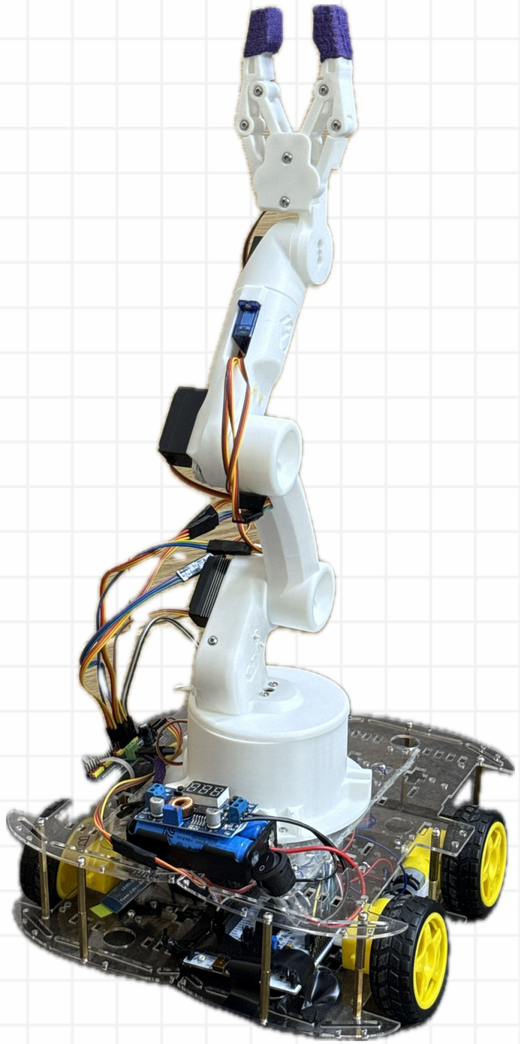
Source: Global Market Insights (GMI), "Mobile Manipulator Market Size & Forecast (2024–2034)"

01 프로젝트 소개



앱 기반 RC 카 및 6 축 로봇팔 제어 시스템

01 프로젝트 소개



- flutter 앱 기반 RC 카 및 6 축 로봇팔 제어
- STM32 의 Timer/PWM 을 활용한 모터 제어
- IMU 센서 융합과 실시간 PID 제어 알고리즘
- 16-Byte 가변 패킷 및 RMW 비트 제어
- Flash Memory 기반의 티칭 데이터 영구 저장

01 프로젝트 소개 _ 역할분배

정승호	조장 및 프로젝트 총괄 , 로봇팔 제어
안재권	앱개발 및 로봇팔 제어
채윤식	RC 카 및 로봇팔 제어 및 하드웨어
전민준	RC 카 및 IMU 기반 PID 제어
이종현	RC 카 및 IMU 기반 PID 제어

01 프로젝트 소개 _ 개발과정

1 단계 : 시스템 설계
- 전체 시스템 설계
- 필요 기능 및 주요 하드웨어 선정
- 통신 프로토콜 및 제어 구조

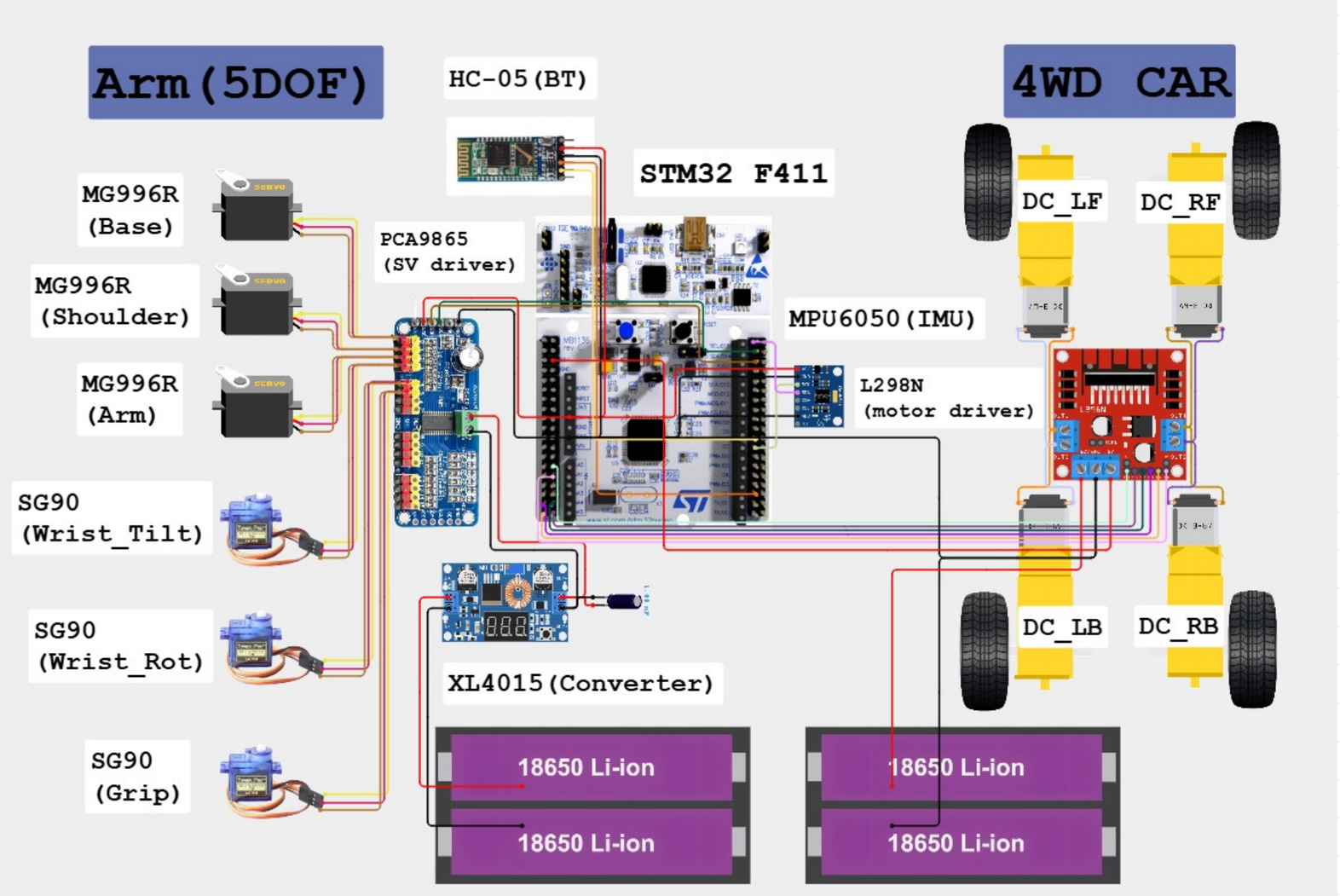
2 단계 : 구현
- 제어 App 개발 (Flutter)
- RC 카 조립 및 관련 기능 구현
- 로봇팔 3D 프린팅 및 조립

3 단계 : 하드웨어 통합
- RC 카 및 로봇팔 모듈 결합
- 기타 구성요소 (전원 / 배선) 조립

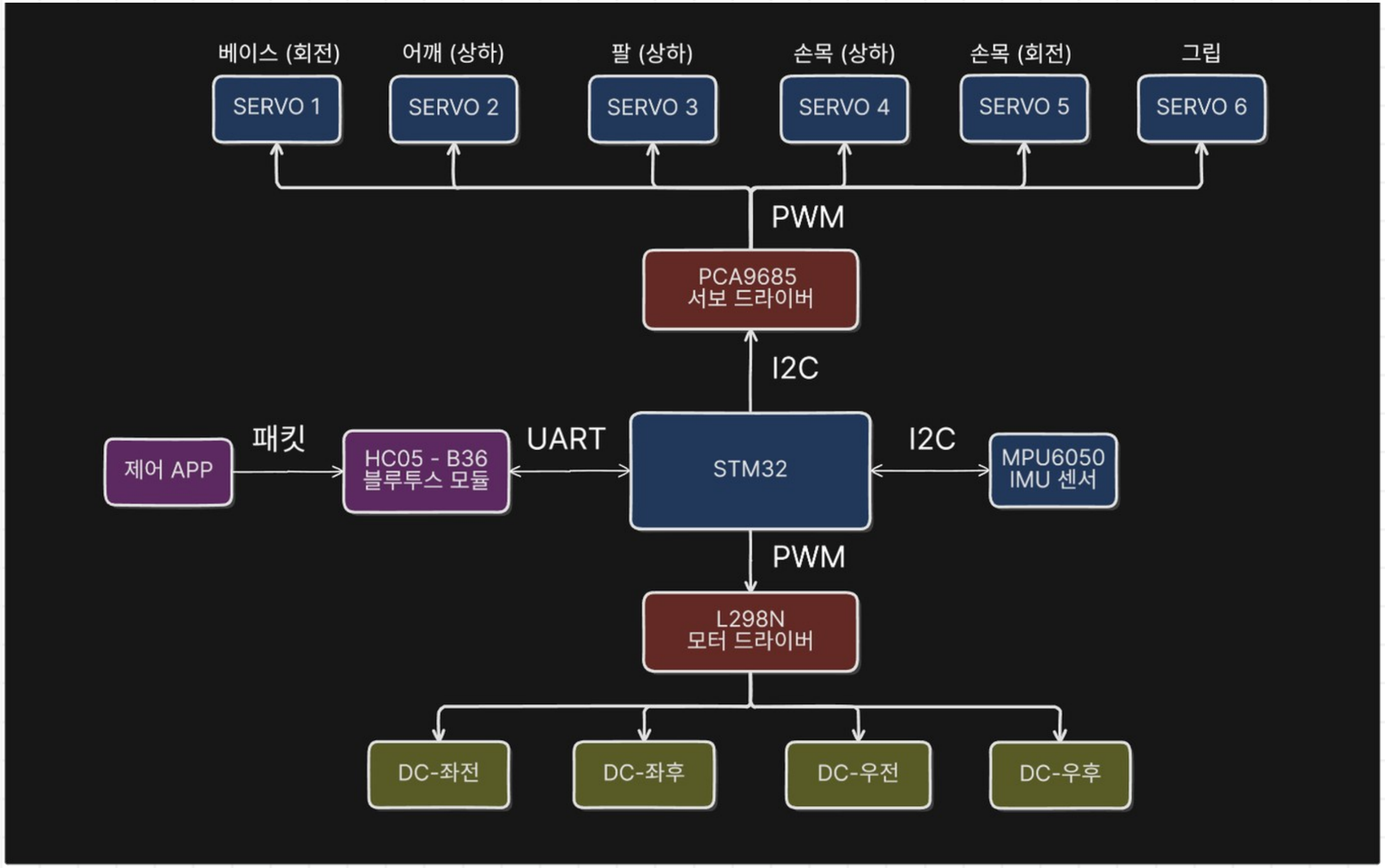
4 단계 : 테스트 및 검증
- PID 제어 및 감도 튜닝
- 로봇팔 WP 티칭 시퀀스 구현
- STM32 플래시 메모리 저장

02 시스템 구성

02 시스템 구성


Arm(5DOF)
HC-05(BT)
4WD CAR
STM32 F411
MG996R (Base)
MG996R (Shoulder)
MG996R (Arm)
PCA9865 (SV driver)
MPU6050(IMU)
L298N (motor driver)
SG90 (Wrist_Tilt)
SG90 (Wrist_Rot)
SG90 (Grip)
XL4015(Converter)
DC_LF
DC_RF
DC_LB
DC_RB
18650 Li-ion
18650 Li-ion
18650 Li-ion
18650 Li-ion

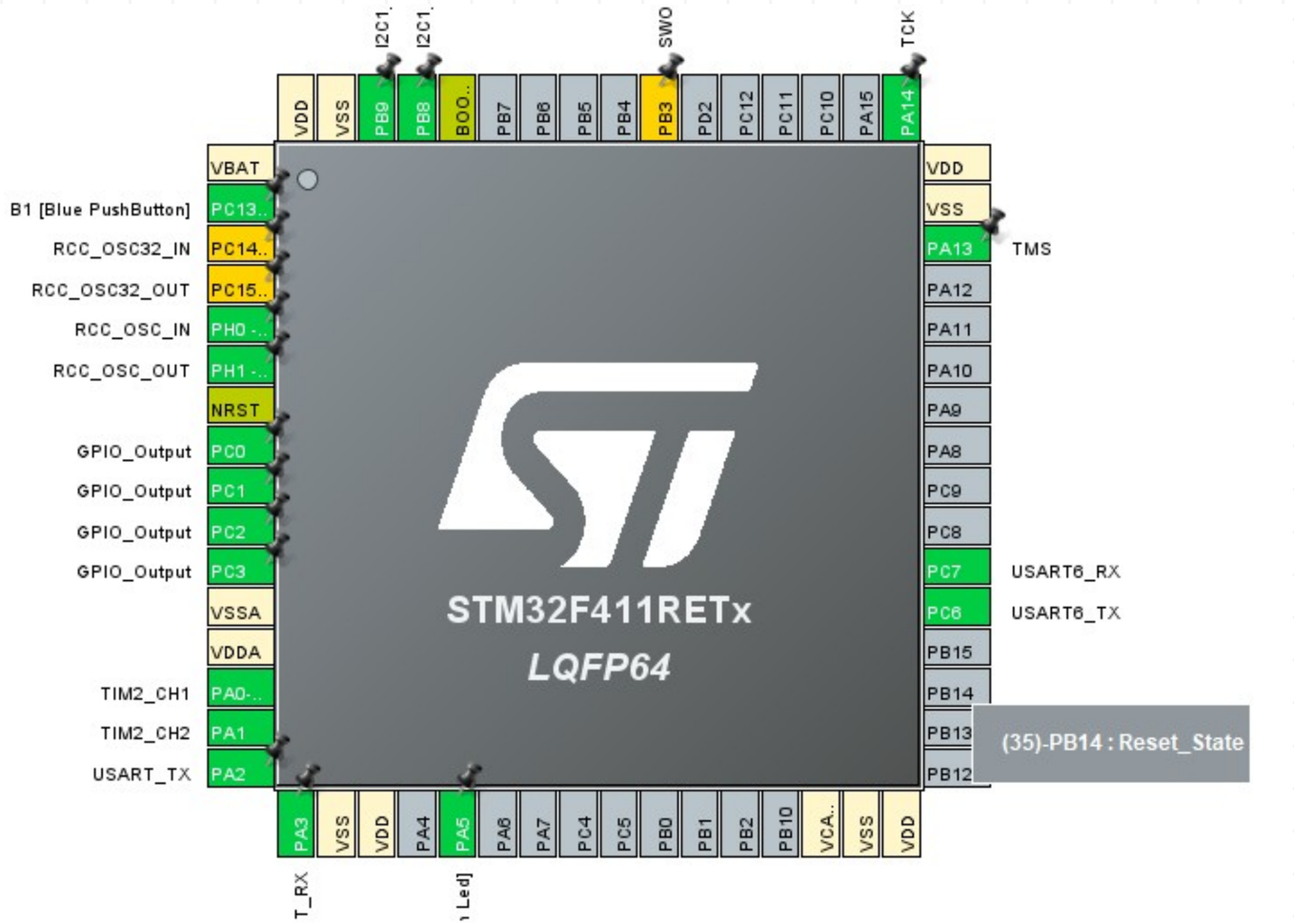
02 시스템 구성 _ 요약


베이스 (회전)
어깨 (상하)
팔 (상하)
손목 (상하)
손목 (회전)
그립
SERVO 1
SERVO 2
SERVO 3
SERVO 4
SERVO 5
SERVO 6
PWM
PCA9685
서보 드라이버
I2C
패킷
UART
I2C
제어 APP
HC05 - B36
블루투스 모듈
STM32
MPU6050
IMU 센서
PWM
L298N
모터 드라이버
DC-좌전
DC-좌후
DC-우전
DC-우후

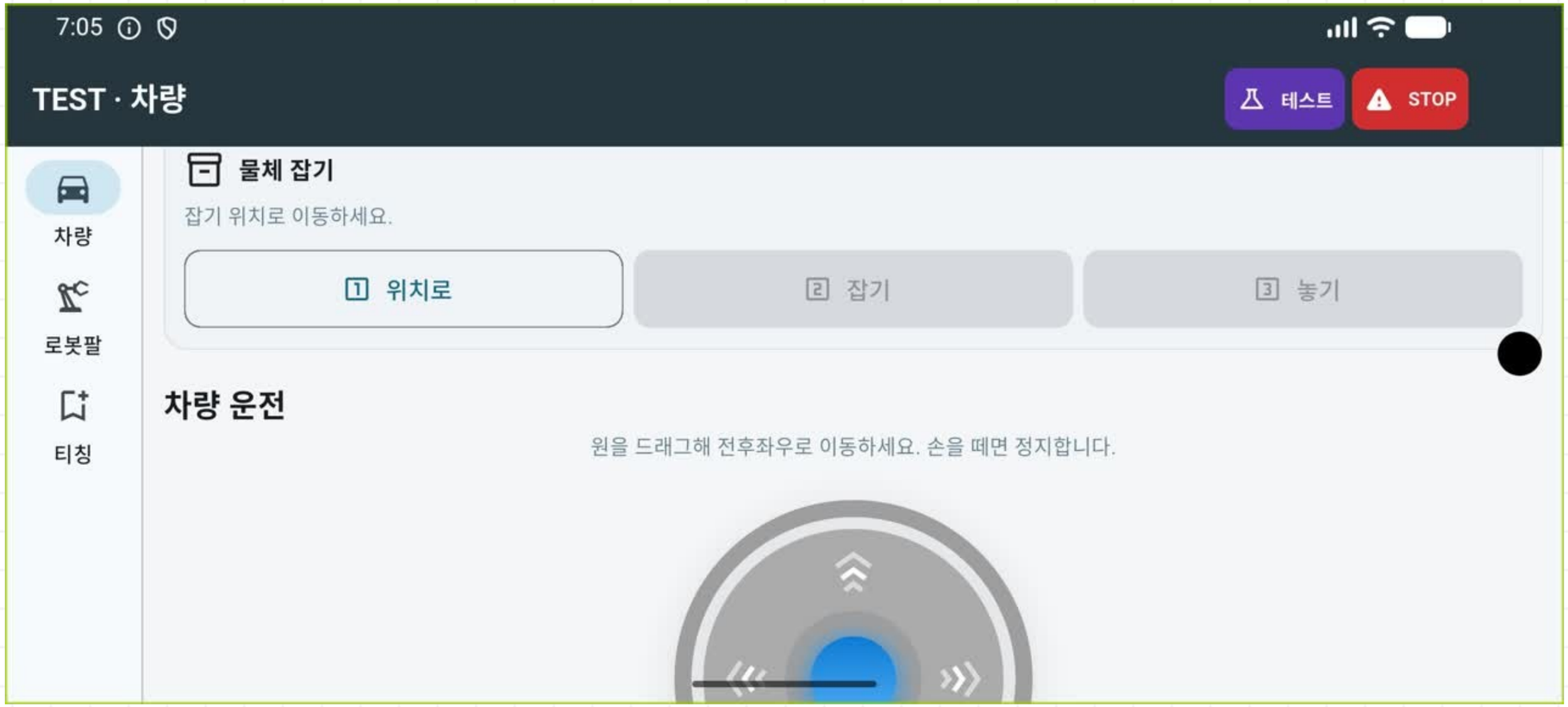
02 시스템 구성 _ 하드웨어스펙

구분	부품	주요 사양
제어부	STM32 NUCLEO-F411RE	ARM Cortex-M4, 100MHz, 512KB Flash
통신	HC-05 블루투스	Classic BT(SPP), UART 9600bps
서보 드라이버	PCA9685	16 채널 PWM, I2C 제어
모터 드라이버	L298N	듀얼 H- 브리지 , 채널당 2A
IMU 센서	MPU6050	6 축 IMU(가속도 + 자이로), I2C
구동 모터	DC 기어드모터 (4EA)	-
관절 서보	MG996R (4EA), SG90 (2EA)	메탈 / 플라스틱 기어 , PWM 각도 제어 (0~180°)
전원	배터리 , 스위치	배터리 2 개 / 2 개사용

02 시스템 구성 _ 핀맵


I2C1..
I2C1..
SWO
TCK
VDD
VSS
PB9
PB8
BOO..
PB7
PB6
PB5
PB4
PB3
PD2
PC12
PC11
PC10
PA15
PA14
VBAT
B1 [Blue PushButton]
PC13..
RCC_OSC32_IN
PC14..
RCC_OSC32_OUT
PC15..
RCC_OSC_IN
PH0 -..
RCC_OSC_OUT
PH1 -..
NRST
GPIO_Output
PC0
GPIO_Output
PC1
GPIO_Output
PC2
GPIO_Output
PC3
VSSA
VDDA
TIM2_CH1
PA0-..
TIM2_CH2
PA1
USART_TX
PA2
VDD
VSS
PA13
TMS
PA12
PA11
PA10
PA9
PA8
PC9
PC8
PC7
USART6_RX
PC6
USART6_TX
PB15
PB14
PB13
PB12
(35)-PB14 : Reset_State
STM32F411RETx
LQFP64
PA3
VSS
VDD
PA4
PA5
PA6
PA7
PC4
PC5
PB0
PB1
PB2
PB10
VCA..
VSS
VDD
T_RX
Led]

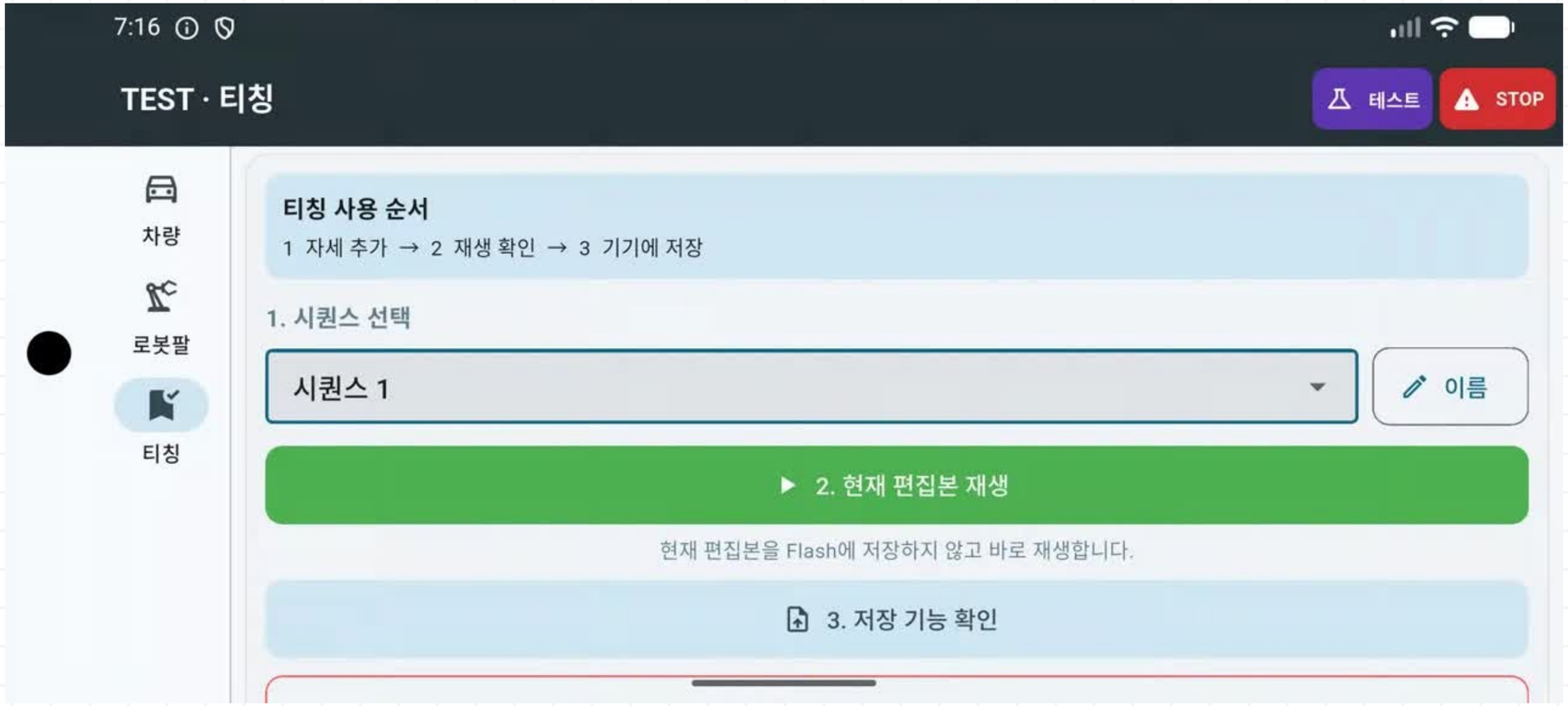
02 시스템 구성 _App(RC 카 제어)



02 시스템 구성 _App(로봇팔 제어)

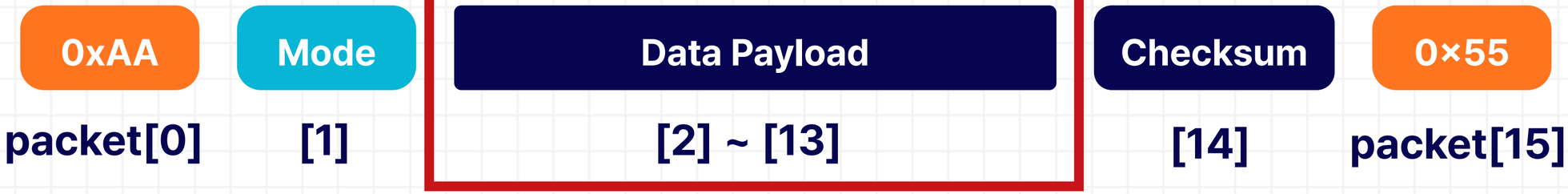


02 시스템 구성 _App(티칭)



02 블루투스 통신 패킷 설명

총 16 바이트 패킷 + 모드별 가변 Payload

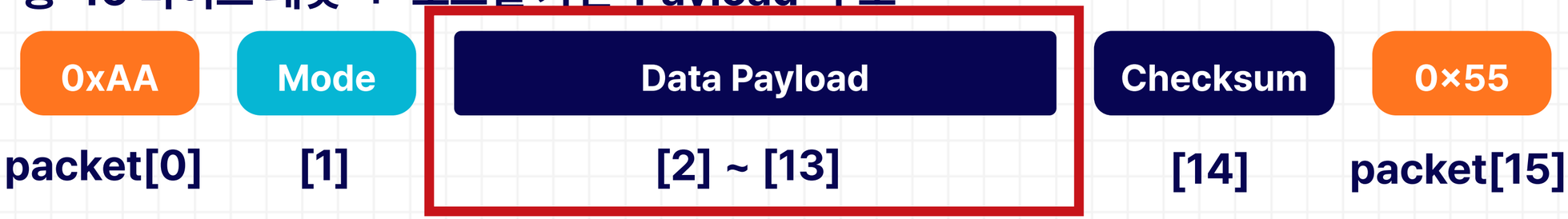


- Mode 0 → 차량 제어
- Mode 1 → 로봇팔 제어
- Mode 2 → 티칭 모드
- Mode 3 → PID, 서보 원점 보정

각 모드별
패킷 구성 차별화

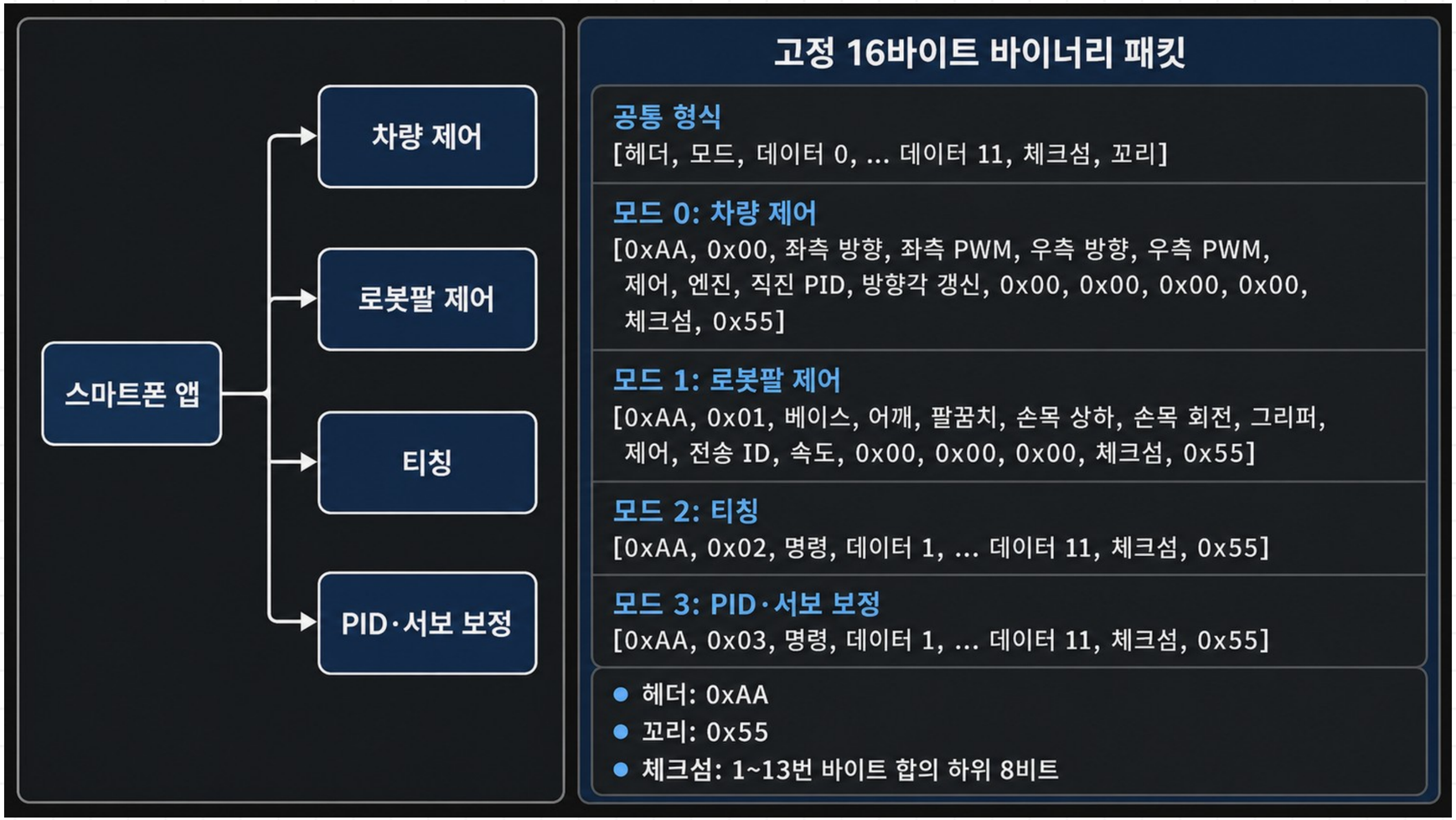
02 블루투스 통신 패킷 설명

총 16 바이트 패킷 + 모드별 가변 Payload 구조



- Mode 0 → 차량 주행 · E-STOP · PID
- Mode 1 → 6 채널 로봇팔 · 속도 · Transaction ID
- Mode 2 → 12 개 티칭 시퀀스 · 조회 · 재생 · 저장
- Mode 3 → PID 계수 · 서보 3 점 보정 · 설정 저장

02 블루투스 통신 패킷 설명



고정 16바이트 바이너리 패킷

공통 형식
[헤더, 모드, 데이터 0, ... 데이터 11, 체크섬, 꼬리]

모드 0: 차량 제어
[0xAA, 0x00, 좌측 방향, 좌측 PWM, 우측 방향, 우측 PWM, 제어, 엔진, 직진 PID, 방향각 갱신, 0x00, 0x00, 0x00, 0x00, 체크섬, 0x55]

모드 1: 로봇팔 제어
[0xAA, 0x01, 베이스, 어깨, 팔꿈치, 손목 상하, 손목 회전, 그리퍼, 제어, 전송 ID, 속도, 0x00, 0x00, 0x00, 체크섬, 0x55]

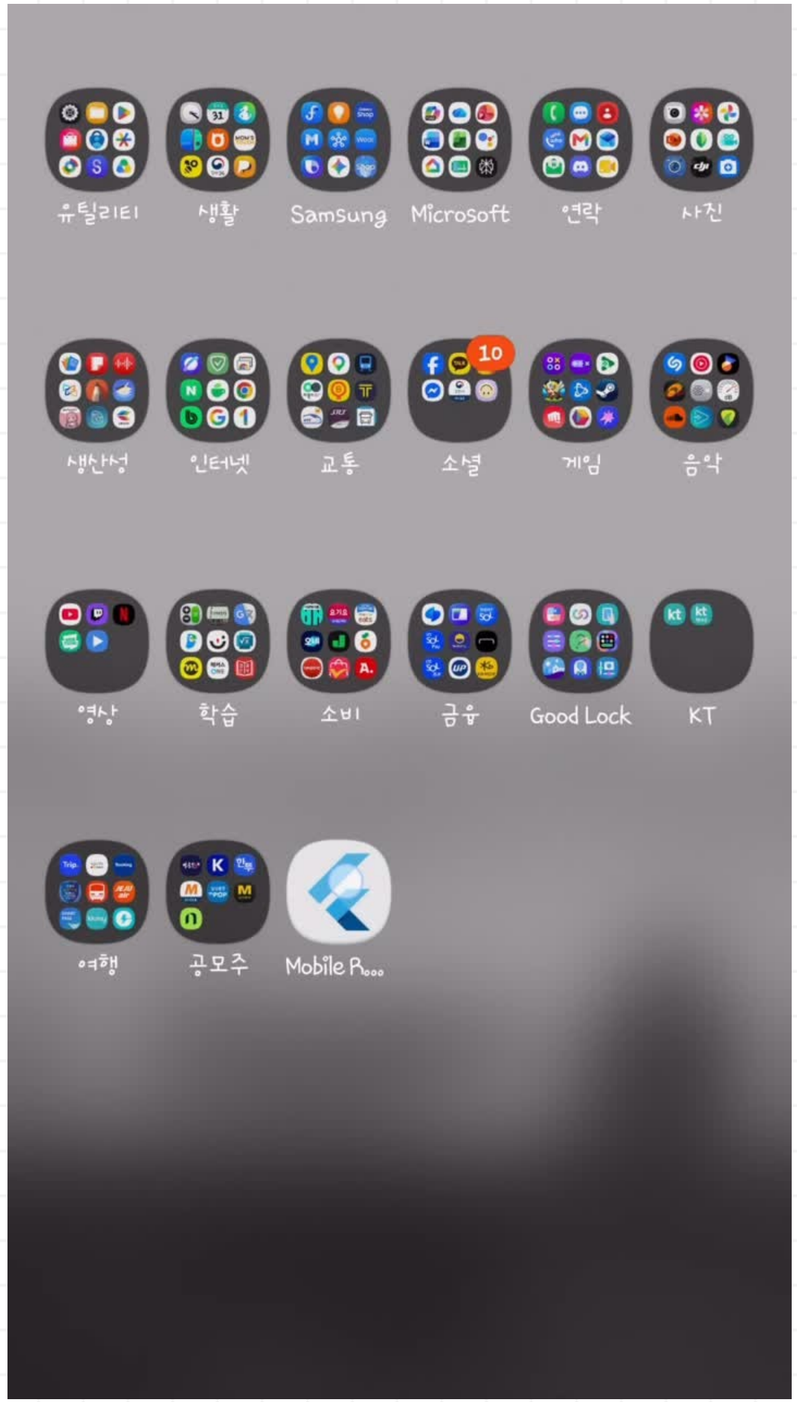
모드 2: 티칭
[0xAA, 0x02, 명령, 데이터 1, ... 데이터 11, 체크섬, 0x55]

모드 3: PID·서보 보정
[0xAA, 0x03, 명령, 데이터 1, ... 데이터 11, 체크섬, 0x55]

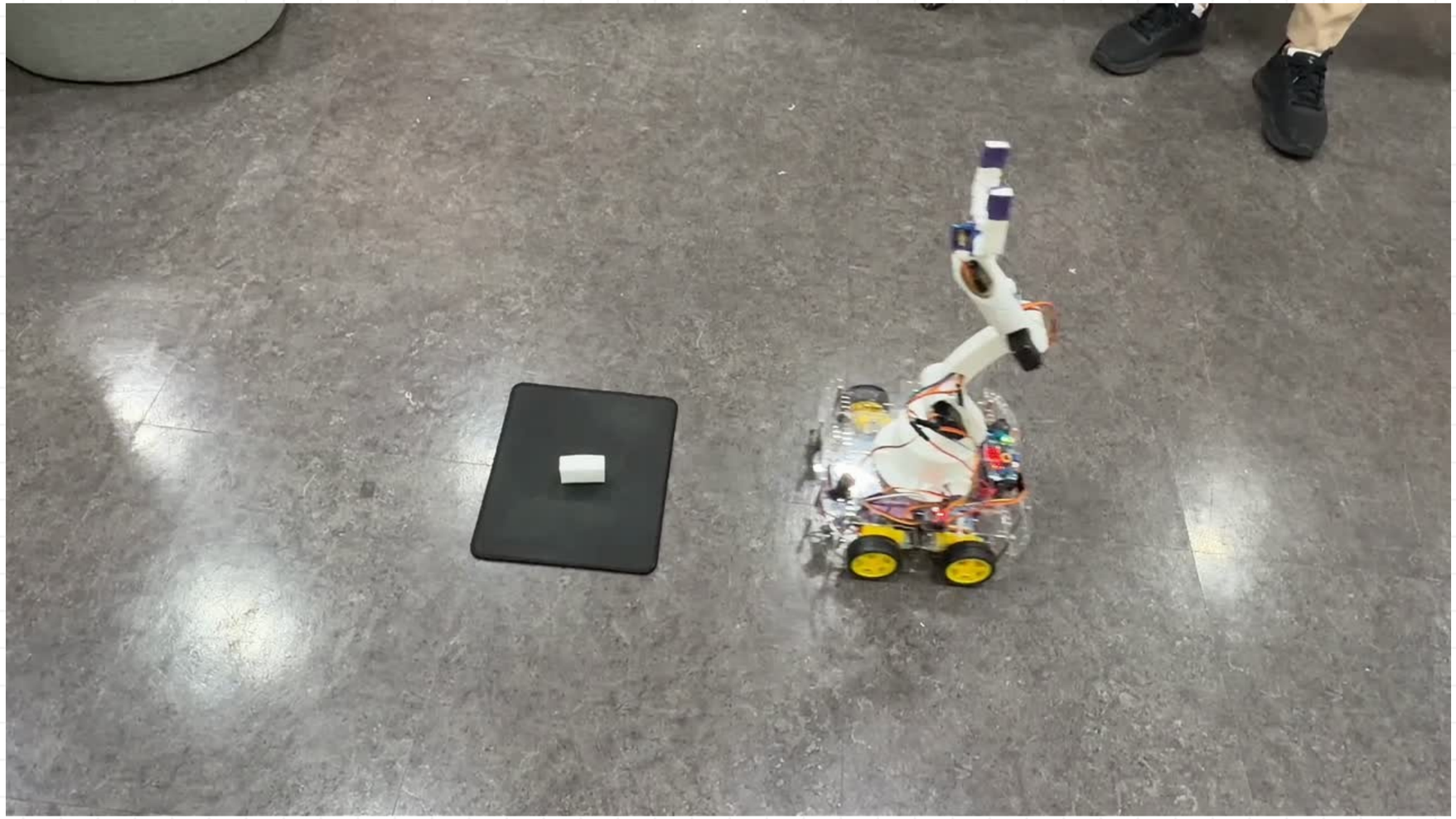
- 헤더: 0xAA
- 꼬리: 0x55
- 체크섬: 1~13번 바이트 합의 하위 8비트

03 시연 영상

03 시연영상 1_App UI



03 시연영상 2_ 간단한 작동



03 시연영상 3_RC 카 주행 (PID)

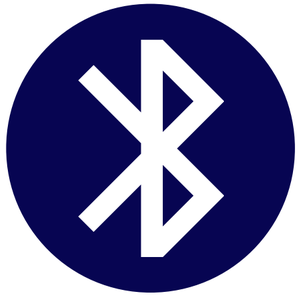


03 시연영상 4_Pick & Place



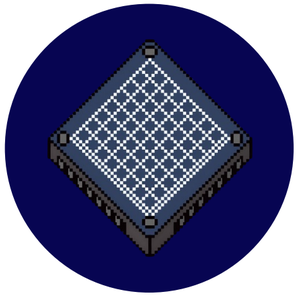
04 핵심 기술

04 RC 카 제어 핵심



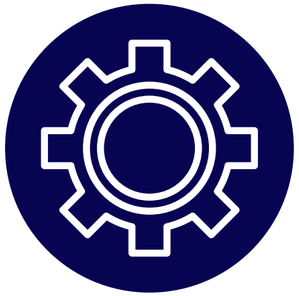
조이스틱 기반

모바일 앱의 조이스틱 값을
블루투스로 받아서 차량이 해야 할
동작을 결정



IMU 기반 방향 측정

MPU6050 의 자이로 센서로 차량이
실제로 얼마나 회전했는지 측정



PID 제어

목표 방향과 IMU 로 측정한 실제
방향의 차이를 이용하여 모터의 속도를
PID 와 PWM 로 조절

04 차량 동작 흐름

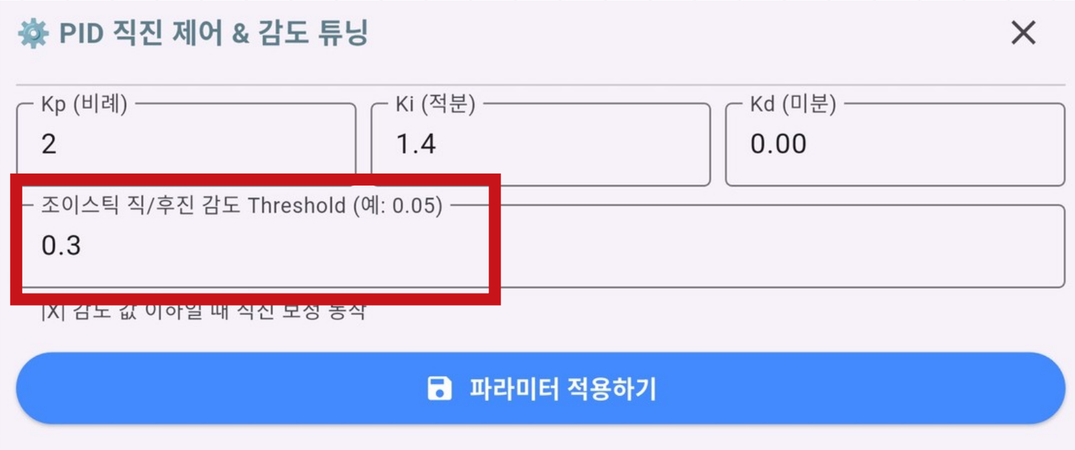
차량 동작 흐름

- 모바일 앱 조이스틱 블루투스 명령 수신
- 직진 PID 활성화 시 현재 Yaw 목표 방향으로 저장
- IMU (MPU6050) 으로 실제 회전 측정
- 목표값과 측정값의 오차 계산
- PID 제어
- PID 출력으로 좌우 모터 PWM 보정

04 IMU 기반 PID 제어

직진 판단 및 목표 방향 설정

조이스틱이 직진 범위에 들어가면 PID 가 활성화되고 , 그 순간 IMU 의 현재 Yaw 를 목표 방향으로 저장





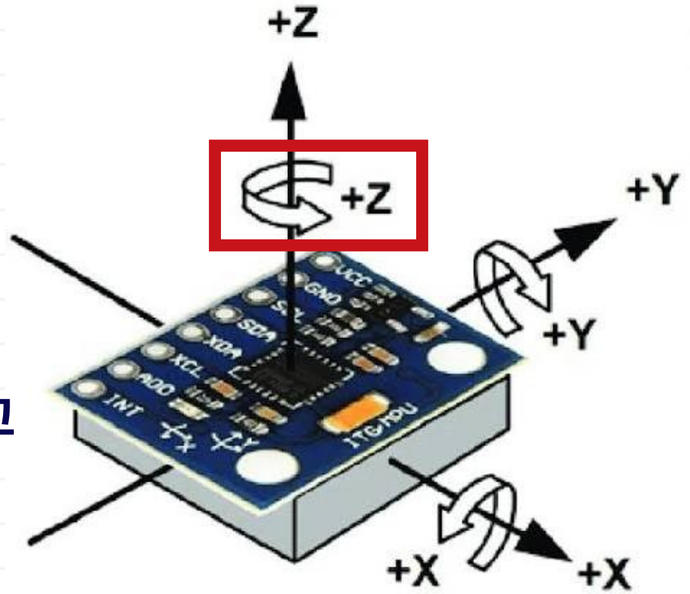
* 회전 조작 중에는 조이스틱 값으로만 모터를 제어하고 , 다시 직진 구간에 들어오면 목표 Yaw 를 새로 설정

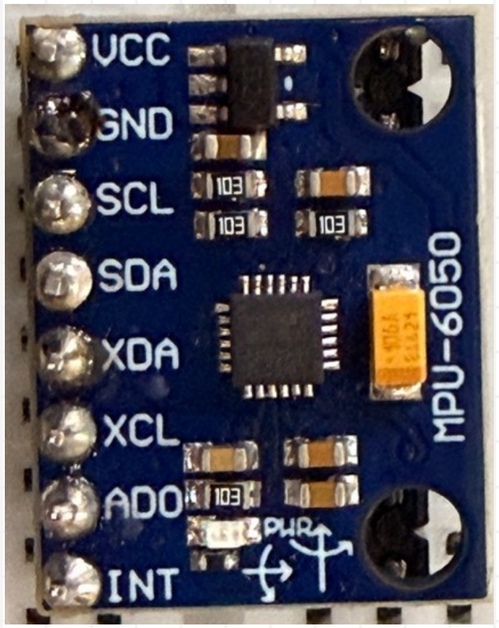
04 IMU 기반 PID 제어

적용 센서

MPU6050 계열 6 축 IMU 센서

3 축 가속도 센서와 3 축 자이로 센서로 구성





적용 목적

IMU 의 자이로 센서로 차량의 회전 상태를 측정하고

PID 제어를 통해 차량이 직진하도록 보정

사용 값

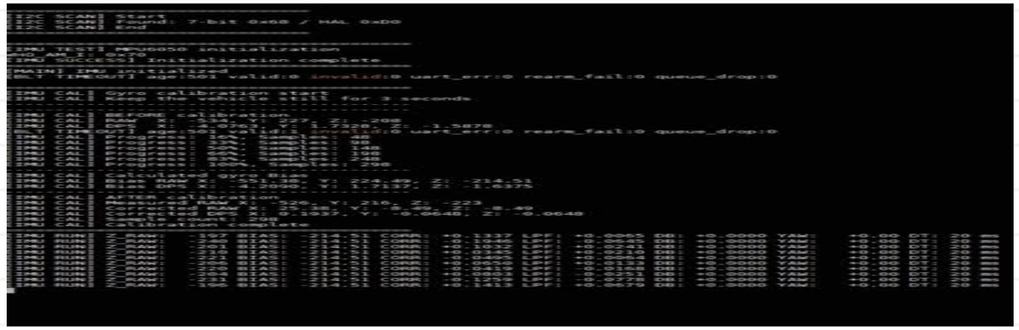
센서의 가속도 3 축과 자이로 3 축을 모두 읽지만 , 차량의 직진 방향 제어에는 Z 축 자이로 각속도만 사용

Z 축 각속도를 시간에 따라 누적하여 현재 Yaw 각도를 계산하고 , PID 제어의 현재 방향으로 사용

04 IMU 기반 PID 제어

IMU 캘리브레이션 , 노이즈 필터링 흐름

- MPU6050 Raw 값 수신
- 3 초 캘리브레이션으로 Bias 계산
- 현재 Raw 값에서 Bias 제거
- 각속도 (dps) 변환
- 1 차 로우패스 필터 적용 (2hz 적용)
- ±0.15dps 데드밴드 적용



Bias 캘리브레이션 : 센서의 일정한 영점 오차 제거

```
[IMU RUN] Z RAW:  -220 BIAS: -214.51 CORR: -0.0419
```

로우패스 필터 : 순간적으로 빠르게 튀는 노이즈 감소

```
LPF: -0.0168
```

데드밴드 : 정지 상태의 미세한 잔여 흔들림 제거

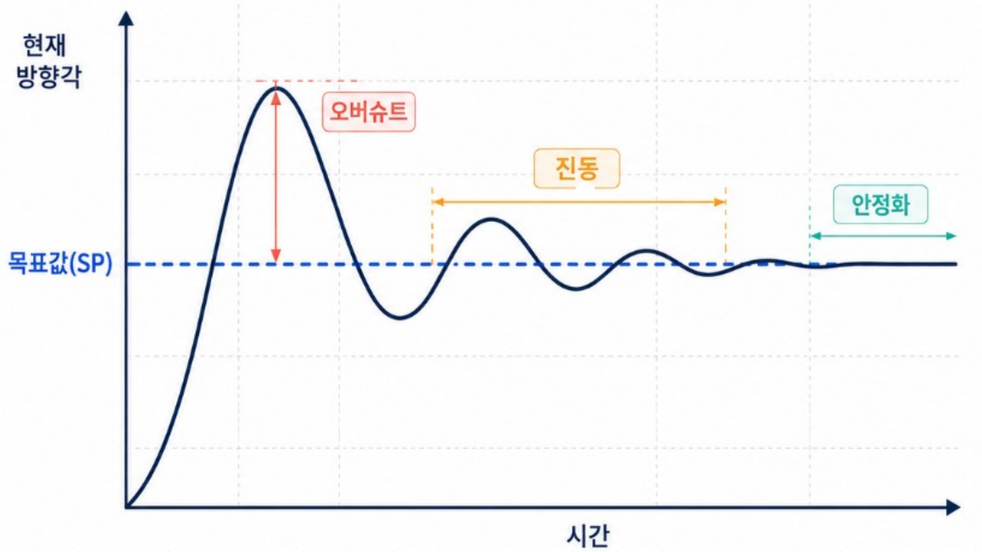
```
DB: +0.0000 YAW:   +0.00
```

*** 캘리브레이션을 하는 이유**

IMU 는 가만히 있어도 센서 오차로 작은 회전값이 발생
캘리브레이션은 이 오차를 기준값으로 저장해 빼줌으로써 정확한 방향을 측정하기 위해 수행

04 IMU 기반 PID 제어

PID



- P(비례 제어): 오차에 비례해 즉시 보정
- I(적분 제어): 계속 쌓이는 작은 오차를 없애 목표 방향에 수렴
- D(미분 제어): 오차가 변하는 속도를 보고 진동과 과도한 보정을 줄임

PID 제어 목적

RC 카는 좌우 모터의 성능 차이 , 바닥 마찰 , 무게중심 등의 영향으로 양쪽 모터에 같은 PWM 을 주어도 한쪽으로 치우칠 수 있음

IMU 센서 (MPU6050) 를 사용해서 차량의 회전 방향인 Yaw 를 측정하고 , 목표 방향과 현재 방향의 차이를 PID 로 계산하여 좌우 모터 PWM 을 자동으로 조절

04 IMU 기반 PID 제어

PID 계수 적용



방향 오차 및 PID 출력 계산

방향 오차 = 목표 Yaw – 현재 Yaw

PID 출력 = 2.00 × 현재 오차 + 1.40 × 오차 누적값

왼쪽 = 198 - 17.89 ≈ 180

오른쪽 = 198 + 17.89 ≈ 216

최종 PWM 보정

(오른쪽으로 틀어짐 → PID 출력 양수 , 왼쪽으로 틀어짐 → PID 출력 음수)

왼쪽 최종 PWM = 조이스틱 왼쪽 PWM - PID 출력

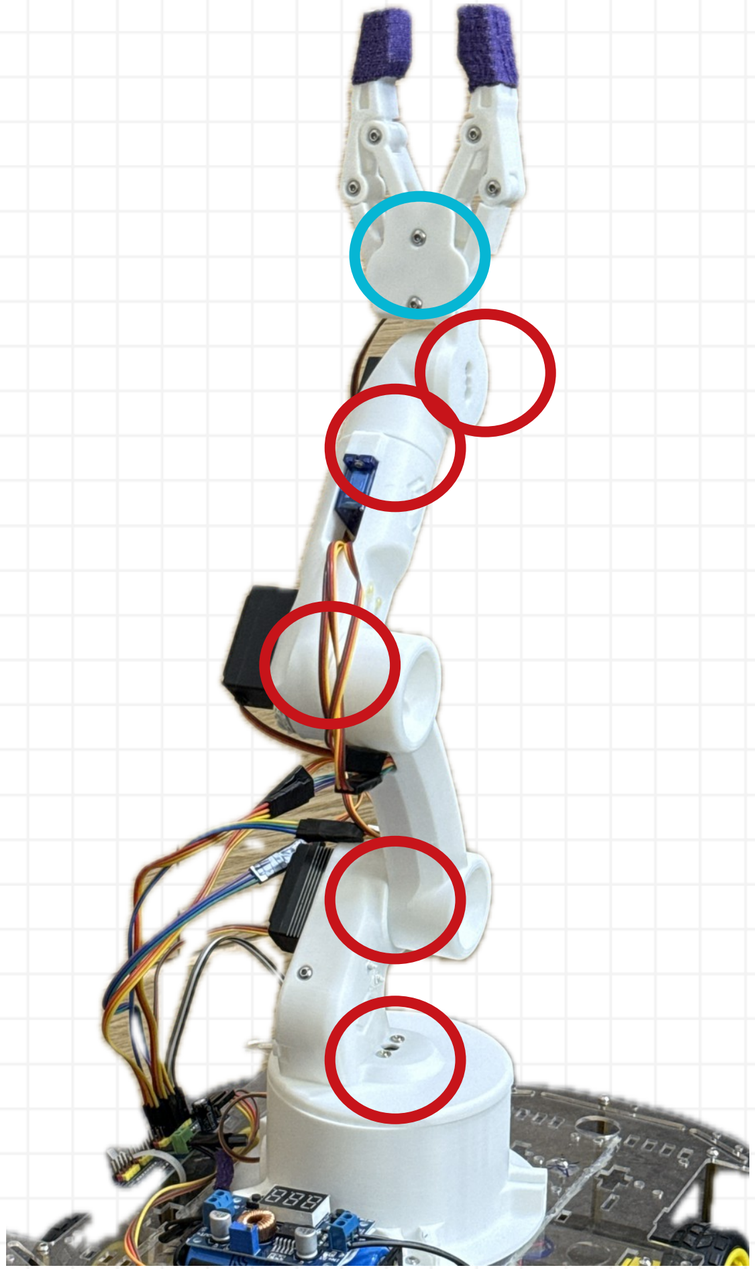
오른쪽 최종 PWM = 조이스틱 오른쪽 PWM + PID 출력

04 Robot-Arm 제어 핵심

관절각 기반 로봇팔 제어

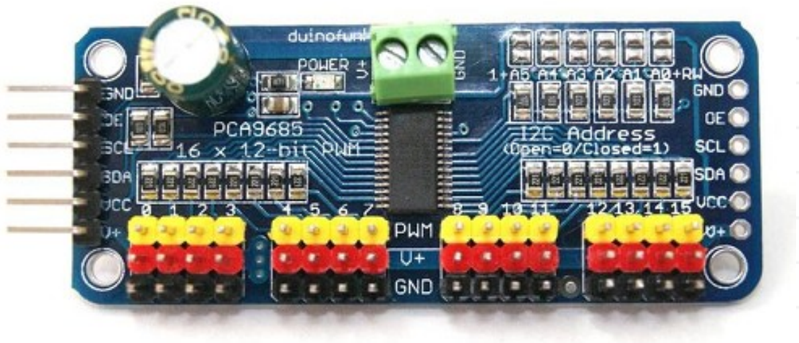
- Gripper (그리퍼)
- Wrist Rotate (손목 회전)
- Wrist Tilt (손목 상하)
- Elbow (팔꿈치)
- Shoulder (어깨)
- Base (베이스)

5 개의 회전 관절과 1 개의 그리퍼를 가진
6 자유도 로봇팔 구성



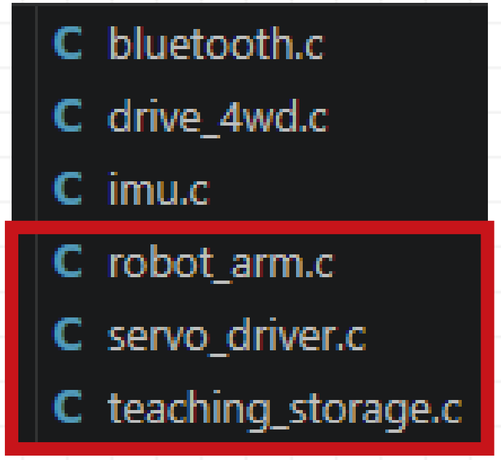
04 Robot-Arm 제어 핵심

6 개의 서보는 개별 PWM 제어가 아닌
PCA9685 를 이용한 6 채널 PWM 제어



보드의 50Hz, 20ms 주기의 PWM 신호를
I2C 통신을 이용해 12 비트 카운트 값으로 전달

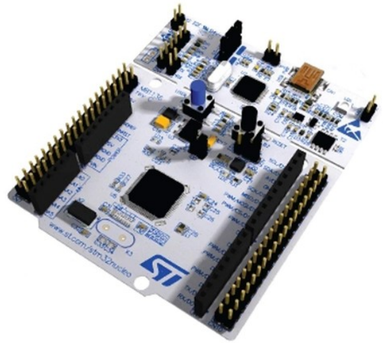
- ST 보드 타이머 채널 최소화
- 6 개 이상의 서보 동일한 주파수 제어 가능
- 각 채널의 펄스폭 개별 설정 가능
- 로봇팔 제어 코드와 하드웨어 PWM 코드를 분리 가능



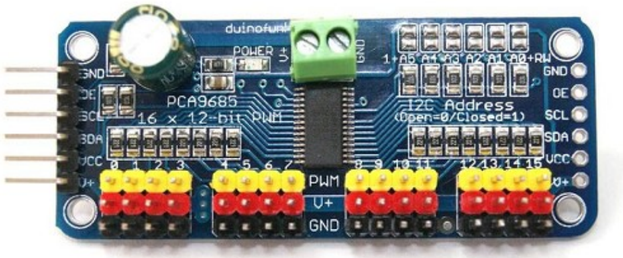
04 Robot-Arm 제어 핵심

PCA9685 를 이용한 6 채널 서보 PWM 제어

- 관절 목표 위치 (어깨 관절 50° 목표값)
- 서보 펄스폭 (μs) 계산 (50° → 1056μs)
- PCA9685 카운트 값 계산 (1056μs →216)
- I2C 레지스터 기록 (어깨가 연결된 2 번채널에 기록)
- 서보 PWM 출력
- 목표 위치로 서보 이동



I2C



PWM



04 Robot-Arm 제어 핵심

관절부는 0° ~ 180° 를 다음과 같이 변환
관절부 : -90°~90° (원점은 0°)

그리퍼는 열림 / 닫힘만을 사용
그리퍼 : 0~100% (개폐 값)

3 점 보정 (-90°/0°/+90°) 으로 원점 / 리미트 설정
실측 펄스 (350μs ~ 2650μs 범위) 저장 가능



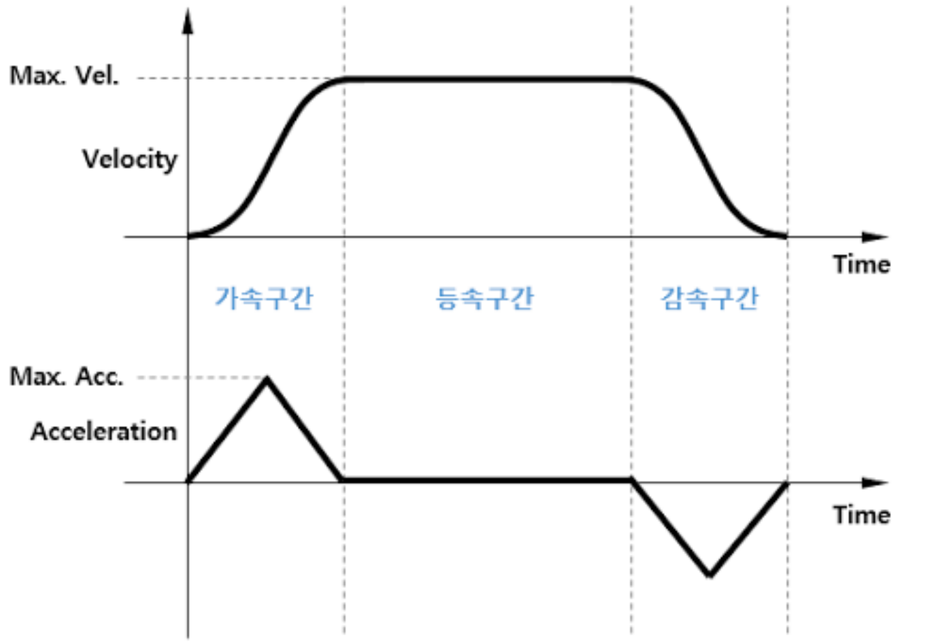
04 Robot-Arm 제어 핵심

단순한 딜레이 방식 속도 조절이 아닌
S - Curve 방식의 속도 조절
(20ms 마다 펄스폭을 단계별 변화)
→ 로봇팔이 부드럽게 작동하도록 !

펄스폭 변화량이
작게 → 크게 → 작게 변화함

서보 속도 조절 가능

- 100%: 최대 약 62.5°/s
- 80%: 최대 약 50°/s
- 50%: 최대 약 31.25°/s

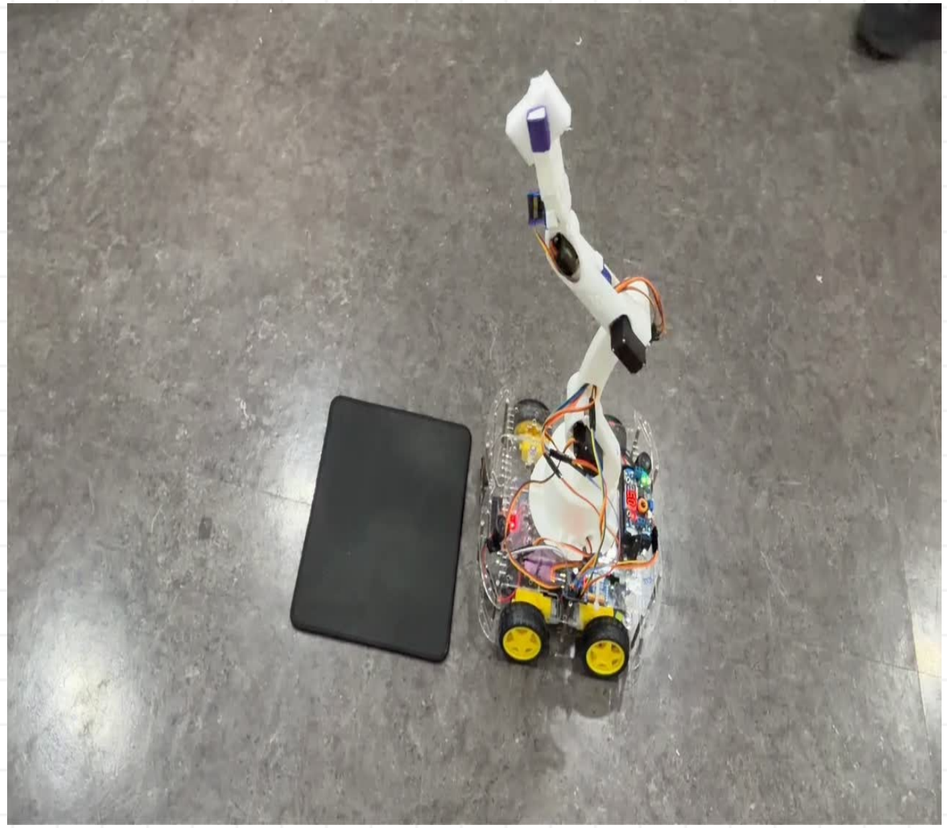


S-형 프로파일 (Sigmoid Curve)

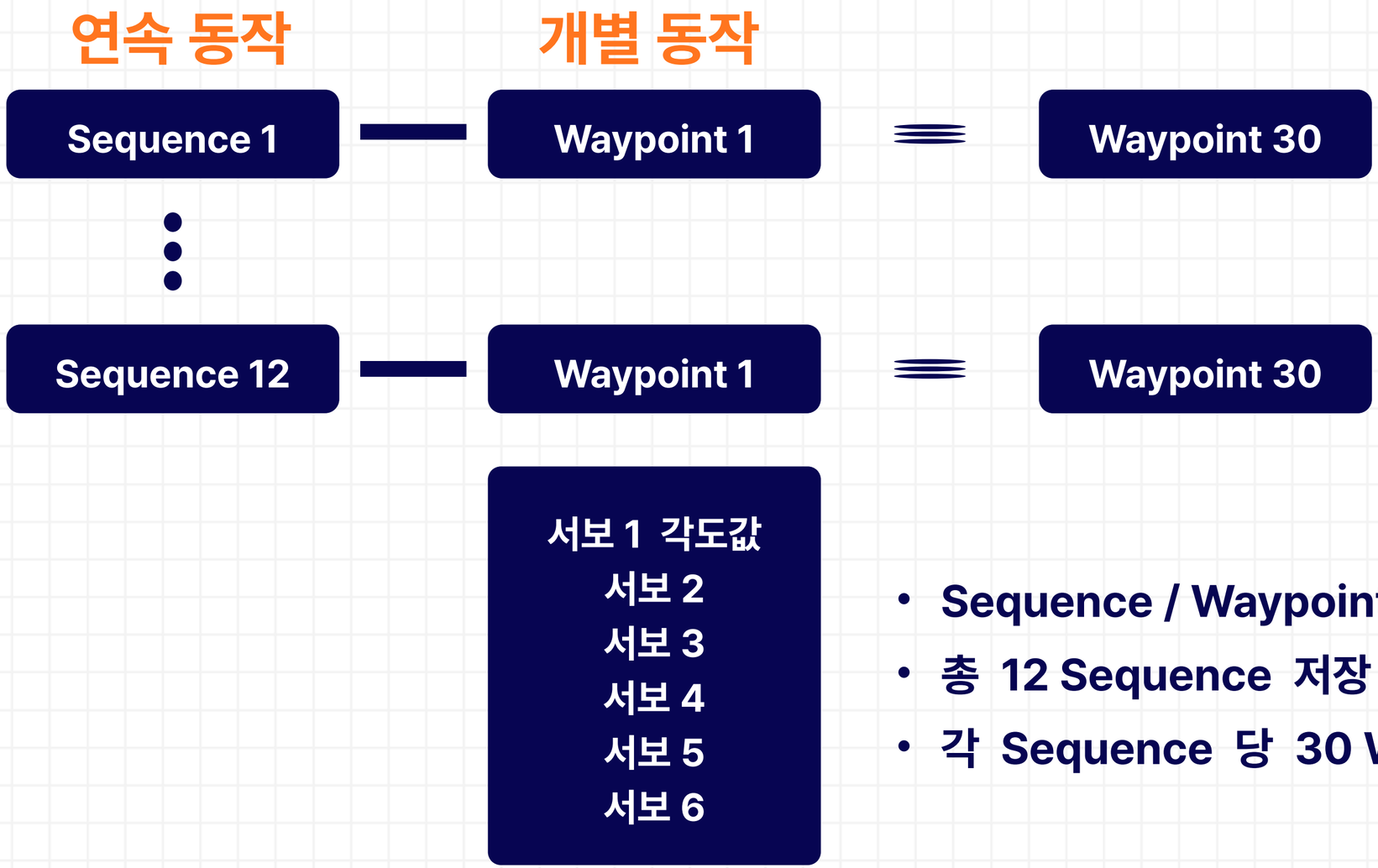
04 Robot-Arm 제어 핵심

PCA9685 를 이용한 6 채널 서보 PWM 제어

- 5 차 다항식 S-Curve 계산
- 20ms 시점의 목표 각도 계산
- 현재 시점의 위치 · 속도 · 가속도 계산
- 계산된 관절 위치를 서보 펄스폭으로 변환
- PCA9685 채널에 새로운 PWM 값 출력
- 목표 위치로 서보 이동



04 Robot-Arm 티칭



- Sequence / Waypoint
- 총 12 Sequence 저장 가능
- 각 Sequence 당 30 Waypoint 저장

04 Robot-Arm 티칭 _RMW 방식

R

READ

Flash 데이터 불러오기
부팅 시 Sector 7 의 설정값과 티칭
데이터를 RAM 으로 읽음

M

MODIFY

RAM 에서 데이터 수정
앱에서 변경한 설정값을 Flash 에
바로 쓰지 않고 RAM 에서 수정

W

WRITE

Flash 에 일괄 저장
저장 버튼을 누르면 수정된 RAM
데이터를 Sector 7 에 한 번에 기록

04 Robot-Arm 티칭

Sector 7:

프로그램 영역과 분리된 128KB Flash 공간

티칭 데이터와 설정값을 전원이 꺼져도

안전하게 보존하기 위해 사용

→ 메모리 오염 최소화

Table 4. Flash module organization (STM32F411xC/E)

Block	Name	Block base addresses	Size
Main memory	Sector 0	0x0800 0000 - 0x0800 3FFF	16 Kbytes
	Sector 1	0x0800 4000 - 0x0800 7FFF	16 Kbytes
	Sector 2	0x0800 8000 - 0x0800 BFFF	16 Kbytes
	Sector 3	0x0800 C000 - 0x0800 FFFF	16 Kbytes
	Sector 4	0x0801 0000 - 0x0801 FFFF	64 Kbytes
	Sector 5	0x0802 0000 - 0x0803 FFFF	128 Kbytes
	Sector 6	0x0804 0000 - 0x0805 FFFF	128 Kbytes
	Sector 7	0x0806 0000 - 0x0807 FFFF	128 Kbytes
System memory		0x1FFF 0000 - 0x1FFF 77FF	30 Kbytes
OTP area		0x1FFF 7800 - 0x1FFF 7A0F	528 bytes
Option bytes		0x1FFF C000 - 0x1FFF C00F	16 bytes

```
Device name : STM32F411xC/E
NVM size    : 512 KBytes
Device type : MCU
Device CPU  : Cortex-M4
```

```
[build] [1/2] Building C object CMakeFiles/mobile_retrieval_robo
[build] [2/2] Linking C executable mobile_retrieval_robot.elf
[build] Memory region         Used Size  Region Size  %age Used
[build]             RAM:       29032 B       128 KB     22.15%
[build]           FLASH:       72272 B       384 KB     18.38%
[driver] Build completed: 00:00:00.988
[build] Build finished with exit code 0
```

04 Robot-Arm 안전 로직

로봇팔의 안전 로직

- 부팅 시 서보 출력 OFF
- 최초 운행 시 원점 정렬 후 활성화
- Flash 작업 중 차량 주행 잠금
- PCA9685 오류 시 출력 차단 후 1 초 간격 재초기화
- 10 초간 로봇팔 응답 없으면 앱에서 E-STOP 요청

04 FreeRTOS 우선순위

FreeRTOS 태스크의 우선순위

driveTask : High	명령 즉시 , 최대 20 ms 대기모터 출력 , PID, 500 ms 통신 타임아웃
bluetoothTask : AboveNormal	5 ms DMA 수신 데이터 파싱 , 명령 큐 전달
armTask : Normal	명령 확인 5 ms, 서보 출력 20 ms S-curve,PCA9685, 티칭 , Flash
imuTask : BelowNormal	고정 20 ms MPU6050 측정 , 필터 , 상대 Yaw 계산
statusTask : Low	250 ms PID 와 IMU 상태를 번갈아 송신

05 문제해결 및 발전

05 문제해결

고속 주행 시 PID 효과

PWM 이 최댓값인 255 에 도달해
PID 보정값이 모터 출력에 미반영

추가 출력 여유가 없어
좌우 모터의 방향 보정이 제한됨

최대 출력을 220 PWM 으로 제한

서보 부하 시 떨림

부하 증가 시 손목 회전과 그리퍼 서보에서 순간적인 떨림 발생

긴 케이블과 커넥터 저항으로 서보 단자 전압이 순간적으로 낮아짐

커패시터 적용 및 전원 배선 점검

로봇팔

목표 각도로 즉시 적용 시 로봇팔에 무리가 감

PWM 펄스폭의 급격한 변경으로 속도와 가속도가 불연속적으로 변함

서보 교체 및 S-Curve 제어

05 향후 발전 과제

- 역기구학 도입을 통한 XYZ 공간 좌표 기반 로봇팔 제어 고도화
- BLE 사용 , 엔코더 모터 사용 등 추가 장비 도입으로 성능 향상
- AI 비전 연동을 통한 객체 인식 및 자율주행 MOMA 구축
- 모듈형 커스텀 설계를 통한 다목적 작업성 확보 (지게차 등)

06 소감

06 소감

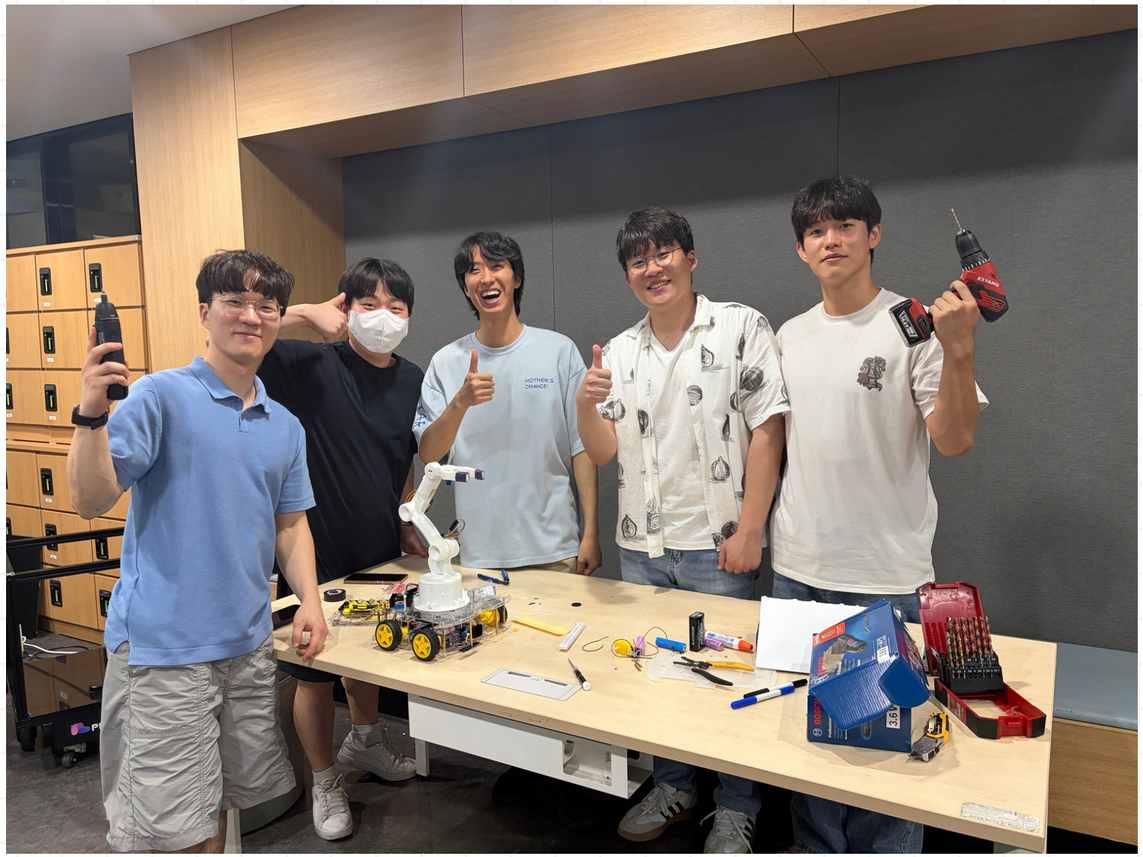
이번 프로젝트를 진행하며 문제가 발생했을 때 , 원인이 소프트웨어인지 하드웨어인지 구분하는 과정이 생각보다 어려웠음 .

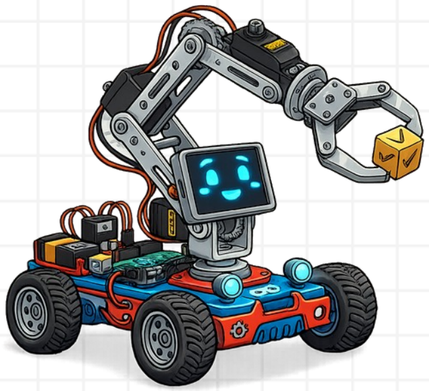
하지만 원인을 하나씩 확인하고 해결하면서 시스템 전체를 바라보는 시야를 넓힐 수 있었음 .
특히 IMU 기반 PID 제어와 로봇팔의 서보 구동을 구현해보며 센서와 제어 기술에 대한 흥미가 커졌음 .

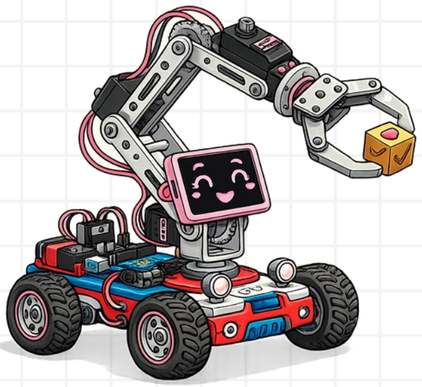
또한 제한된 환경에서 소프트웨어와 하드웨어를 통합하고 성능을 개선하는 과정에서 큰 보람을 느꼈으며 , 앞으로 더 다양한 센서를 활용한 제어 시스템을 만들어 보고 싶어졌음 .

Q&A

잘 들어주셔서
감사합니다 .







TEAM : 차팔이

정승호 전민준 이종현 채윤식 안재권